

DC モータの制御

DC モータとは直流電源を与えることによって駆動するアクチュエータの一種であり, 簡単に扱えることから様々な場面で用いられる.

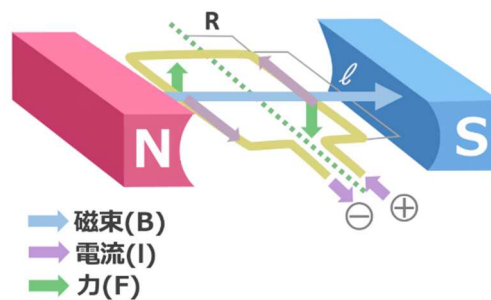
動作原理は非常に単純で, 磁束 B の磁界中に配置されているコイルに電流 I が流れると, フレミングの左手の法則に基づいて力 F (ローレンツ力) が生じる. この力 F を用いてコイル全体を回転させることでアクチュエータとしての駆動力を生み出す.

このとき, コイルの長さを l , コイルの回転半径を R , コイルの回転角度を θ とすると, 力 F およびトルク τ は以下のようにあらわすことができる.

$$\text{力 } F = \text{電流 } I \times \text{磁束 } B$$

$$\text{トルク } \tau = 2 \times \text{力 } F \times \text{コイル長さ } l \times \text{回転半径 } R \times \sin \theta$$

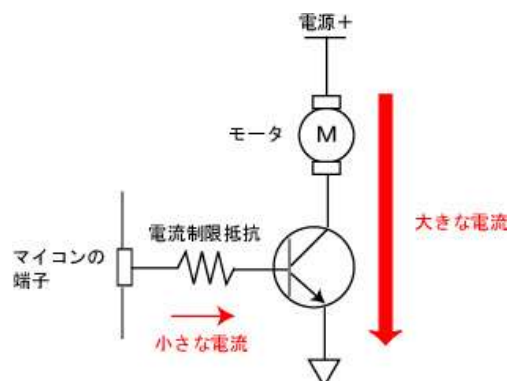
$$= 2 \times \text{電流 } I \times \text{磁束 } B \times \text{コイル長さ } l \times \text{回転半径 } R \times \sin \theta$$



上記の関係式からもわかるように DC モータの力 F およびトルク τ はコイルに流す電流 I に比例することから, その駆動には多くの電流が必要である.

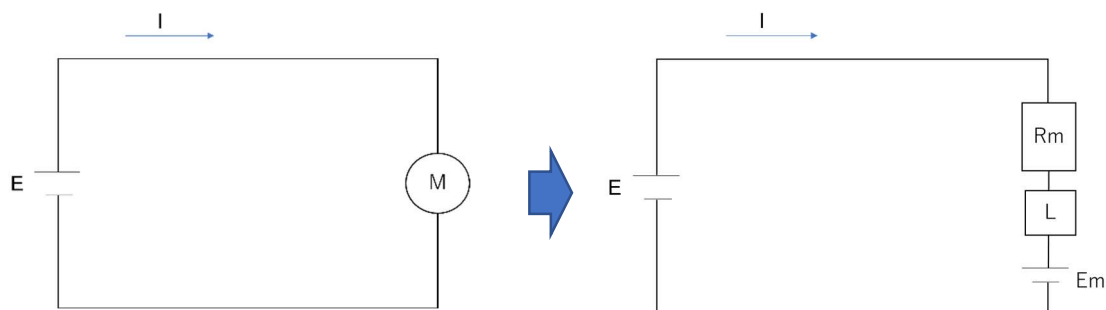
しかしながら, Arduino のようなマイコンの I/O ポートは大量の電流を取り出すことができず, 直接モータと I/O ポートを接続した場合は過大に電流を流そうとするため, マイコンが焼損するなどの故障につながってしまう.

そこでモータに大きい電流を渡すために「トランジスタ」を接続することで, マイコンの I/O から電流を流すのではなく, トランジスタの回路から電流を取り出すことを考える.



ここでモータの内部の構造に着目すると、モータは内部にコイルおよび磁石を備えておりコイルが回転運動をしていることから、内部のコイルの運動により電磁誘導の法則が作用しているとみなすことができる。（モータはそれ自身が発電機として作用する）

モータがある電源 E に接続されている回路を考えると、モータの抵抗 R_m と電磁誘導の法則により生じた起電力 E_m を考えると以下のような等価回路に置き換えることができる。



よって以下のような関係式が導きだせる。

$$E = I \times R_m + E_m$$

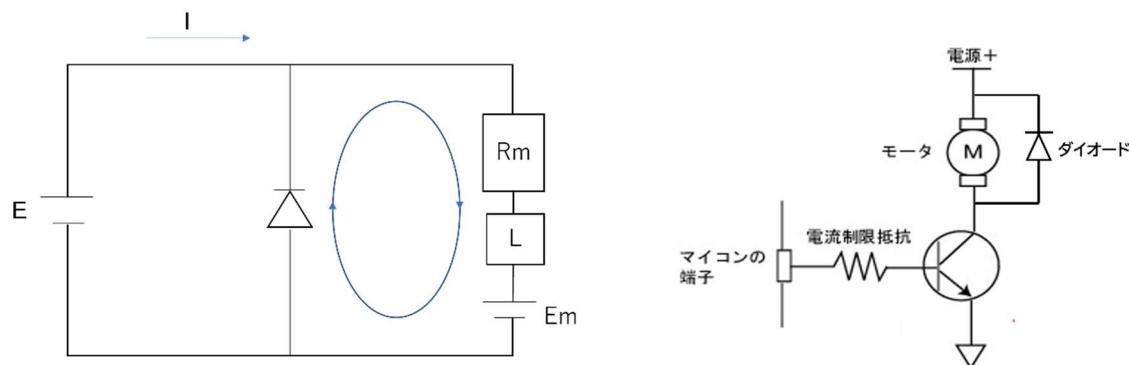
この式は

$$E - E_m = I \times R_m$$

と変形できることから、この E_m はモータの駆動力を打ち消すように作用しているとみなすことができる。このような起電力のことを「逆起電力」と呼ぶ。

モータに供給する電源を OFF にしてその回転を止めた場合、エネルギー保存則よりこの E_m は何らかの形で消費される必要があるが、上記の回路ではこれを消費する術がなく、そのまま大元の電源やトランジスタに戻ってしまい電源回路などを破壊してエネルギーを消費するということになる。

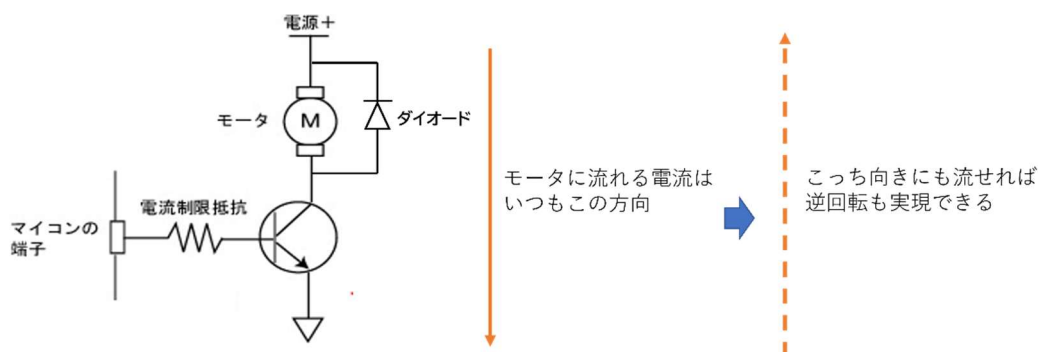
これを解決するために逆起電力が電源回路側に戻らないように逃げ道を作ることによって解消するという方法を考えることができる。その方法のひとつとして、一方向にのみ電気を流すことができる「ダイオード」をモータと並列に配置することで逆起電力をモータのコイルに循環させる方法をとる。これによりコイルの熱としてエネルギーを消費させ全体のエネルギー量を保存して回路を保護することができる。



このほかにもツェナーダイオードを用いて別の回路に逃がして保護する方法やコンデンサに逆起電力を一時的に貯めること（回路内に別に充電電池を用意するような感じ）で保護する方法などがある。

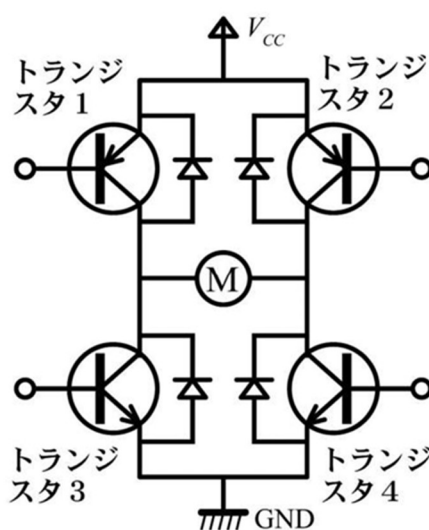
さて、上述した回路によりマイコンの端子から小さな電流を出力することで、モータを電氣的に安全に動作させることができるわけであるが、この回路では常に電流が流れる方向が一定であるため、モータは常に同じ方向にしか回転できない。

モータを逆方向にも回転させるためにはモータの回路に流れる電流の向きを逆方向にも流すことができるようなものがあればよいという発想に行き着く。



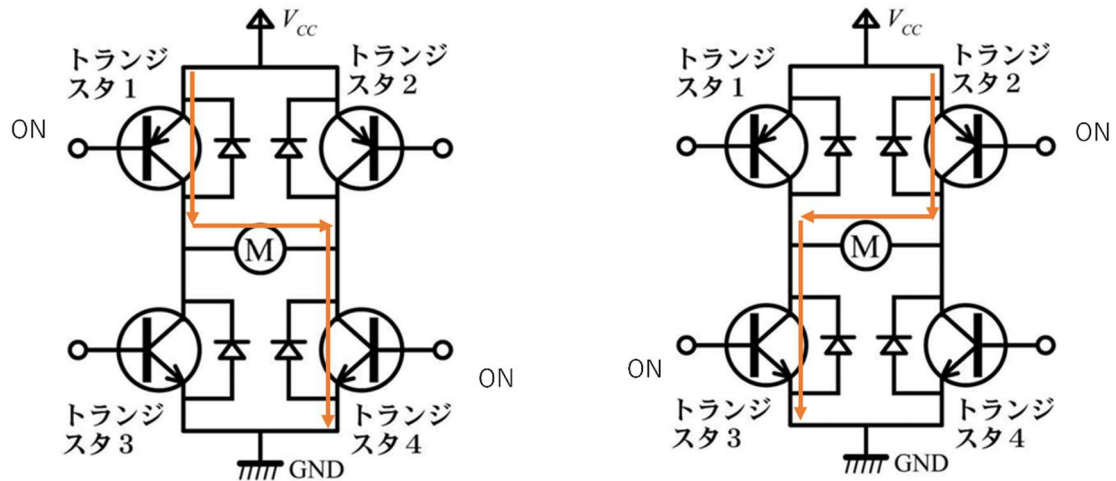
ここでトランジスタの特性に着目すると、電流を大きな値に増幅するだけでなく電気を流すかどうかというスイッチの性質も持っているため、これをうまくスイッチとして活用することでモータに流れる電気の向きを制御できるはずである。

そこでトランジスタを下図のように4つ配置してブリッジ回路を構成することを考える。



この回路において、トランジスタの1番とトランジスタの4番をONにして電気を流すと、その流れは下図左のようにモータの回路に対しては左から右に電流が流れることになる。

一方でトランジスタの2番とトランジスタの3番をONにして電気を流すと、先ほどとは逆に下図右のようにモータの回路に対して左から右に電流が流れることになる。

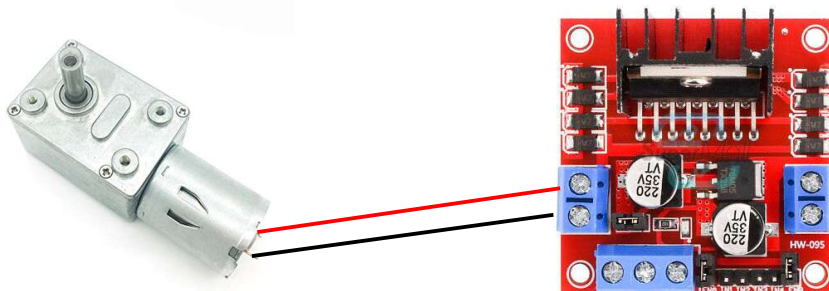


このようにトランジスタを ON にする組み合わせを変えれば左右どちら向きにもモータに電流を流すことができ、その回転方向を切り替えることが可能となる。このような回路のことを「モータドライバ」と呼ぶ。

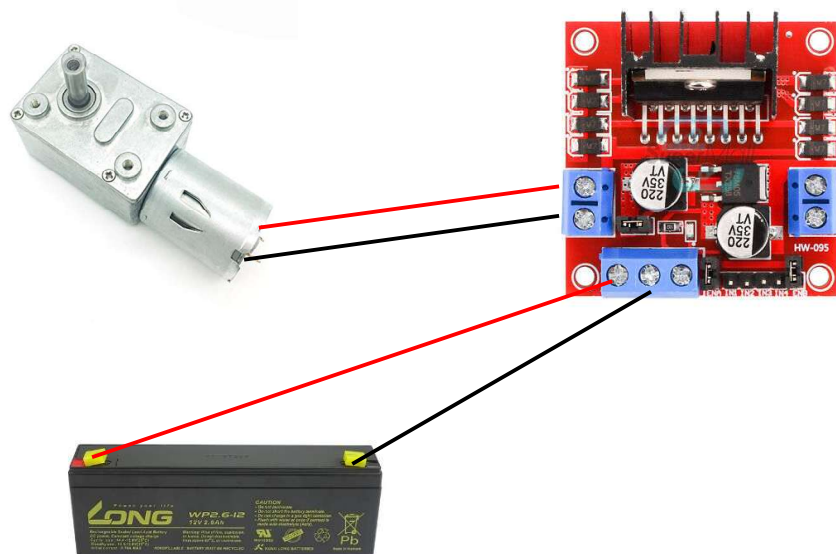
モータドライバは対応する電圧、電流値などが異なる様々なものが流通しているため、各自が用いるモータの仕様（定格電圧や電流）に基づいて適切なものを選定する。今回は定格 12V, 0.18A の DC モータに対応可能なモータドライバとして「L298」を選定して使用することとする。

Arduino とモータの配線

本ドライバは 2 つの出力ポートをもっており DC モータなら 2 つまで制御することができる。今回は DC モータがひとつであるため（本番は 2 つ用いる場合もある）、図のようにモータとドライバの出力ポートを接続する。なお今回は時間の都合上直接ターミナルブロックにネジ止めで固定することとする。（本番時での接続方法については追って資料で提示する）

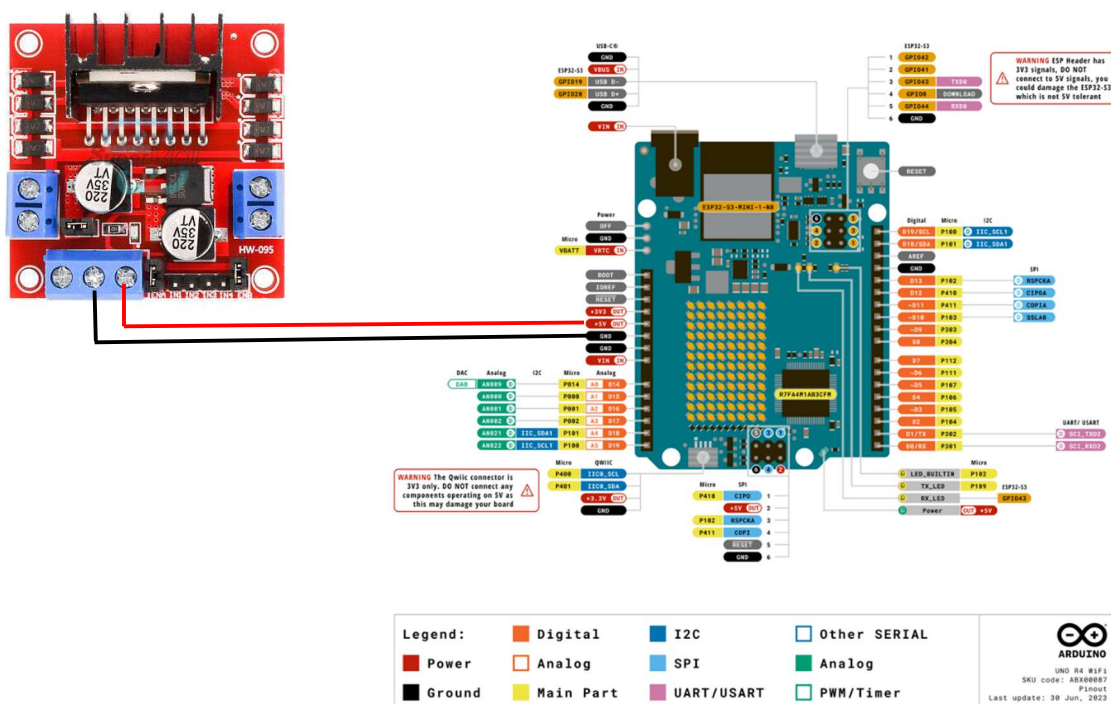


さらにモータを駆動するための電源（12V）が必要であるため、これをドライバに接続する。今回は12Vの電源として鉛蓄電池を使用する。今回は時間の都合上直接ターミナルブロックのネジ止め配線とワニクリップで配線することとする。（本番はスイッチの追加など適切な配線とし、詳細は追って提示する）



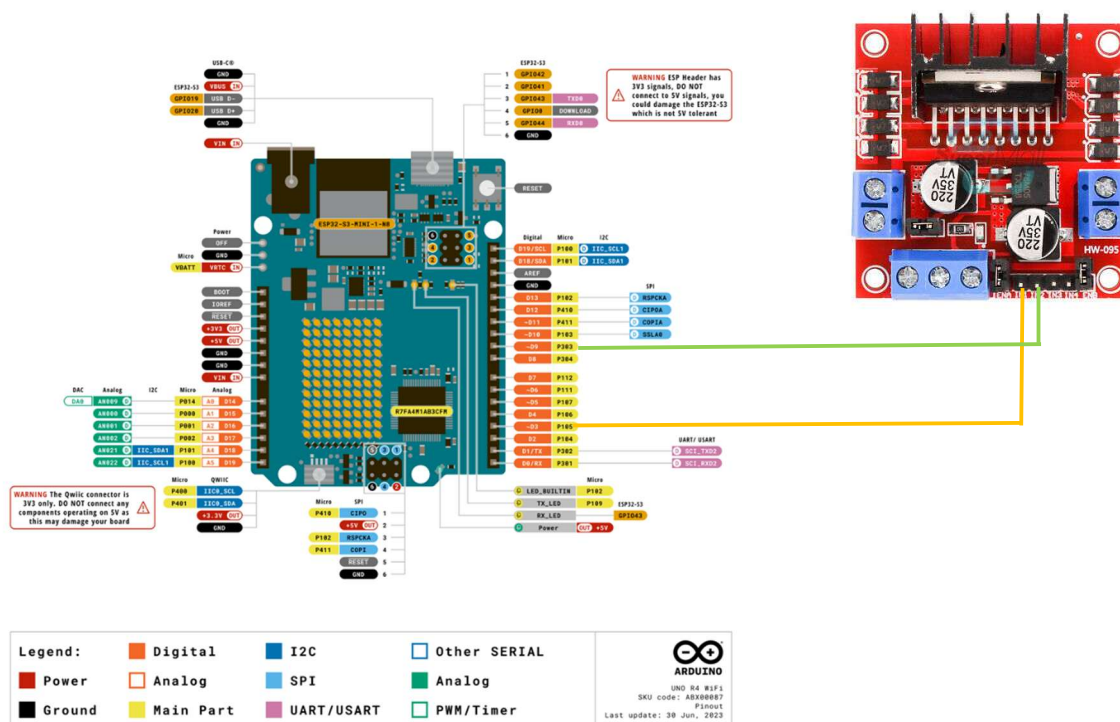
Arduino とモータドライバの配線

モータドライバ自身が駆動するための電源（ロジック電源）として5Vが必要であるため、今回はArduinoの5V出力ピンから供給することとする。先のモータの配線に加えて図のようにモータドライバとArduinoを配線する。（下図はモータ配線を省略）



モータの制御信号を受け取るポートとしてドライバの下部に「IN1」,「IN2」のポートがあるため、これと Arduino の D3,D9（理由は後述）をオス-メスのジャンパケーブルを用いて接続する。

（下図はモータ配線およびロジック電源の配線は省略）



モータ制御プログラム

```
1  int IN1 = 3; //IN1としてD3ピンを接続
2  int IN2 = 9; //IN2としてD9ピンを接続
3
4  void setup() {
5      //ピンモードの設定
6      pinMode(IN1, OUTPUT);
7      pinMode(IN2, OUTPUT);
8  }
9
10 void loop() {
11     //モータ正転(cw)
12     digitalWrite(IN1,HIGH);
13     digitalWrite(IN2,LOW);
14     delay(3000);
15     //モータ惰性回転（慣性で回転させる）
16     digitalWrite(IN1,LOW);
17     digitalWrite(IN2,LOW);
18     delay(3000);
19     //モータ停止（ブレーキ）
20     digitalWrite(IN1,HIGH);
21     digitalWrite(IN2,HIGH);
22     delay(3000);
23     //モータ逆回転(ccw)
24     digitalWrite(IN1,LOW);
25     digitalWrite(IN2,HIGH);
26     delay(3000);
27 }
```

まず、ピンの番号を IN1 = D3, IN2 = D9 と定義し、pinMode をそれぞれ出力ピンとして設定する。

L298 は

- ・ IN1 に HIGH, IN2 に LOW の信号を入力すると正転(CW)する.
- ・ IN1 に LOW, IN2 に HIGH の信号を入力すると逆転(CCW)する.

ただし注意したい点としてモータの回転に伴い慣性モーメントが回転軸および車輪などの回転体にかかるため、回転方向をいきなり反対方向にすることは望ましくない。(車で走行中にいきなりバックすることは困難であることとおなじようなもの)

L298 は

- ・ IN1 と IN2 にそれぞれ LOW の信号を入力すると回転信号を取り止め惰性回転するので、ゆっくり回転スピードを落としてから
- ・ IN1 と IN2 にそれぞれ HIGH の信号を入力すると強制ブレーキするので、回転を止めてから逆回転するようにする.

信号とモータの挙動の対応表は下記のとおりである

	IN1 = HIGH	IN1 = LOW
IN2 = HIGH	ブレーキ=強制停止	逆転(CCW)
IN2 = LOW	正転(CW)	モータ停止=脱力=惰性回転

PWM 信号によるモータ制御

さて、上記のプログラムで2つのピンの信号を切り替えることでモータの回転方向を制御できることは確認できた。ただし、この方法ではモータの回転方向は制御できても回転速度は常に最大角速度で一定のみである。

モータの起電力 V は下記の電磁誘導の式から磁束の時間変化として下記のようにあらわすことができる。

$$V = -n \frac{d\phi}{dt}$$

この式より考察すると、磁束の時間変化(磁束の変化速度)が大きくなるほど起電力の絶対値は大きくなる、DC モータの場合、磁束の変化速度はモータ内部のコイル(もしくは磁石)の回転速度に依存するので、起電力は回転速度に比例すると考えることができる。これよりある比例定数 K_e (起電力定数) を考えると、モータの回転速度(角速度) ω と起電力 V の間には以下の関係式を考えることができる。

$$V = K_e \omega$$

当たり前であるが、この式よりモータの回転速度は起電力が大きくなるほど早くなると考えることができる。つまり電圧の値を調整することでモータの回転速度を制御することができるはずである。よって次はどのようにすれば電圧の値を調整できるか検討する。

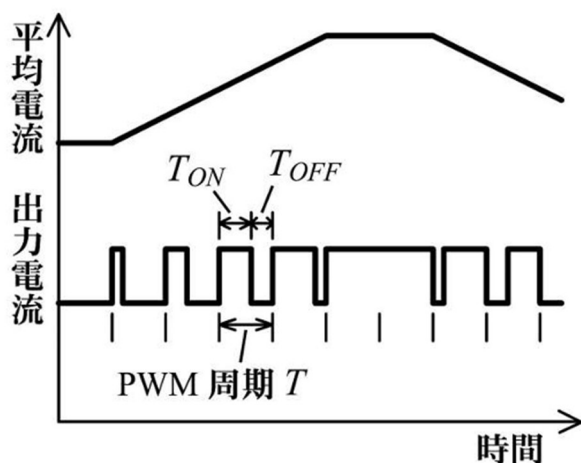
ここでもう一度モータドライバを構成するトランジスタの特性を考えると、これはスイッチの特性を持っているので、ここでトランジスタのスイッチを高速で ON-OFF することを試みる。

すると、ON にしている時間が短い（一瞬で OFF にする）とモータ周りの回路に流れる平均的な電流量は少なくなり、ON にしている時間が長い（スイッチつけっぱなし）と平均的な電流量は多くなる。

（水道の蛇口をひねる時間を想像するとわかりやすいかもしれない。ひねる時間が短いと出る水量は少なく、ひねる時間が長いと水量は多くなる。）

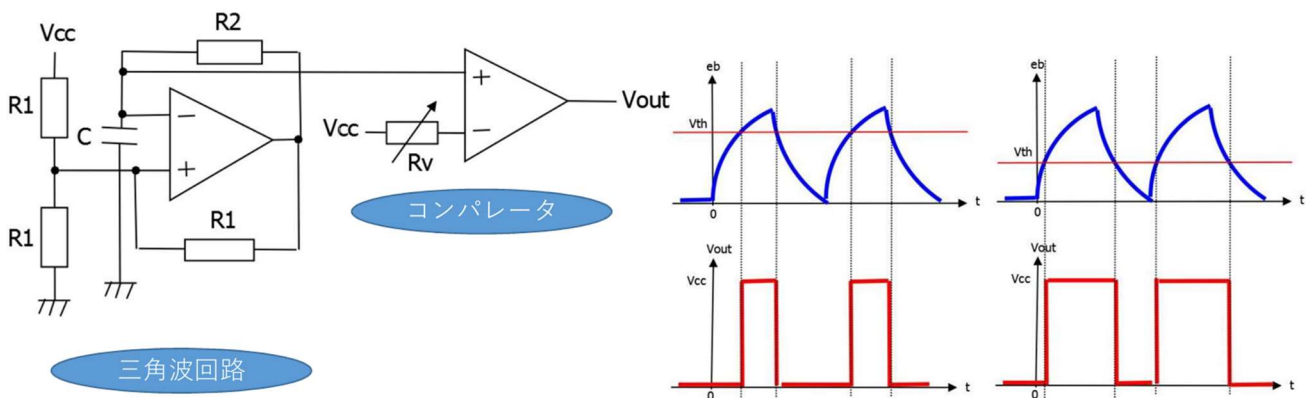
これをある時間幅を 1 周期 T として切り取ると、その周期の範囲内において電流値が決まるので、トランジスタの ON-OFF 信号の時間割合で電流値をコントロールすることができる。

この ON-OFF 信号のことを PWM (pulse wide modulation = パルス幅変調) と呼び、1 周期における ON の時間の割合のことを duty 比と呼ぶ。1 周期で常に ON なら $\text{duty}=100\%$ 、常に OFF なら $\text{duty}=0\%$ 。



オームの法則 $V=IR$ より、電流値と電圧値は比例するので PWM で平均電流値を制御すれば自ずと電圧の値も調整することができることが分かる。これによりモータの回転速度を制御することができる。

PWM は三角波回路とコンパレータにより簡単に生成することができるので、モータの制御方式のひとつとして広く用いられている。



Arduino はこの PWM 信号を出力できる I/O ピンが 6 個備わっており、D3,D5,D6,D9,D10,D11 のピンが該当する。(以前も説明したが~がついているピンがそれ)

ただし、それぞれのピンが出力できる PWM の周期は以下のように異なるので、注意が必要である。

D3, D9, D10, D11 = 480Hz

D5, D6 = 960Hz

よって今回のように 2 つの信号線 (IN1,IN2) でモータを制御する場合は同じ周期の PWM 信号を用いることが望ましいので、IN1=D3, IN2=D9 とする。

PWM によるモータ制御プログラム

```
1  int IN1 = 3; //IN1としてD3ピンを接続
2  int IN2 = 9; //IN2としてD9ピンを接続
3
4  void setup() {
5      //ピンモードの設定
6      pinMode(IN1, OUTPUT);
7      pinMode(IN2, OUTPUT);
8  }
9
10 void loop() {
11     //モータ正転(cw)
12     analogWrite(IN1,255);//duty=100%
13     analogWrite(IN2,0);//duty=0%
14     delay(1000);
15     analogWrite(IN1,128);//duty=約50%
16     analogWrite(IN2,0);//duty=0%
17     delay(1000);
18     analogWrite(IN1,64);//duty=約25%
19     analogWrite(IN2,0);//duty=0%
20     delay(1000);
21     analogWrite(IN1,32);//duty=約12.5%
22     analogWrite(IN2,0);//duty=0%
23     delay(1000);
24     analogWrite(IN1,0);//duty=0%
25     analogWrite(IN2,0);//duty=0%
26     delay(1000);
27 }
28
```

先の例とほぼ変わらないが、信号を与える関数が digitalWrite() から analogWrite() に変化。

また duty 比は 100 分率を 8bit 階調で表現するため、255 で 100% となる点に注意。

練習問題として以下の 3 パターンのプログラムを作成してみよう。

- ・ 逆転させる
- ・ なめらかに速度変化させる
- ・ 徐々に速くなり、その後徐々に遅くなるといった感じに速度変化させる

ネットワーク機能について

今回用いる arduino R4 wifi はその名前のおり無線 LAN モジュールを搭載しており、ネットワークに接続することが可能である。ここではネットワークの接続方法とネットワーク越しにモータの回転方向を制御する例について紹介する。

まず必要なヘッダと変数を定義する。

```
1  #include <WiFiS3.h>
2
3  char ssid[] = "SSIDを記載"; // WiFiルーターに接続する際の SSID (name) を入力。
4  char pass[] = "パスワード"; // WiFiルーターに接続する際の password を入力。
5  int status = WL_IDLE_STATUS; // WiFiの状態
6  IPAddress ip;
```

MAC アドレスを記載する関数を定義

取得した配列を複数の文字毎に分割して表示

```
8  //MACアドレス取得
9  void printMacAddress(byte mac[]) {
10     for (int i = 5; i >= 0; i--) {
11         if (mac[i] < 16) {
12             Serial.print("0");
13         }
14         Serial.print(mac[i], HEX);
15         if (i > 0) {
16             Serial.print(":");
17         }
18     }
19     Serial.println();
20 }
```

WiFi の情報を表示する関数を定義

```
22 // WiFi接続情報の表示関数
23 void printWifiData() {
24     // SSIDの表示
25     Serial.print("SSID: ");
26     Serial.println(WiFi.SSID());
27     //ルーターのmacアドレス
28     byte bssid[6];
29     WiFi.BSSID(bssid);
30     Serial.print("BSSID: ");
31     printMacAddress(bssid);
32     //MACアドレス
33     byte mac[6];
34     WiFi.macAddress(mac);
35     Serial.print("MAC address: ");
36     printMacAddress(mac);
37     //IPアドレス
38     ip = WiFi.localIP();
39     Serial.print("IP Address: ");
40     Serial.println(ip);
41 }
```

setup 関数にて接続などの作業を実行

```
43 void setup() {
44     Serial.begin(115200);
45     delay(1500);
46
47     //WiFiモジュールのファーム確認
48     String fv = WiFi.firmwareVersion();
49
50     // WiFi接続
51     Serial.print("次のSSIDに接続中: ");
52     Serial.println(ssid); // WiFi接続に使うSSIDを表示
53     status = WiFi.begin(ssid, pass); // WiFi接続
54
55     //接続が確立するまで待機
56     while (status != WL_CONNECTED) {
57         delay(500);
58         Serial.print(".");
59     }
60
61     // 接続完了
62     Serial.println("接続完了");
63     //関数呼び出して接続情報を表示
64     printWifiData();
65 }
```

Loop 関数では接続をしてることを確認したら切断作業を実施するだけとする。
実際はここでいろいろネットワーク越しにデータのやりとりをする。

```
67 void loop() {
68     //WiFi接続中なら
69     if (status == WL_CONNECTED) {
70         //WiFi切断
71         Serial.println("WiFi切断中");
72         WiFi.disconnect();
73         // WiFi.statusがWL_IDLE_STATUSになるまで待機
74         while (WiFi.status() != WL_IDLE_STATUS) {
75             delay(500);
76             Serial.print(".");
77         }
78         status = WiFi.status();
79         // Wi-Fi切断結果をシリアルモニタへ出力
80         Serial.println("切断完了");
81     }
}
```

実行すると以下のようにシリアルモニタにて接続先の情報を表示して切断する様子が確認できる。

```
次のSSIDに接続中: AQUOS_R9_8385
接続完了
SSID: AQUOSR9_8385
BSSID: A8:D4:19:A7:AB:BA
MAC address: A4:02:9F:FA:12:F4
IP Address: 192.168.236.69
WiFi切断中
切断完了
```

簡易サーバを Arduino に立ててモータを制御してみる.

先のコードにモータのピン番号の変数を追加

またサーバのクラスを追加

```
1  #include <WiFiS3.h>
2
3  char ssid[] = "SSIDを入力"; // WiFiルーターに接続する際の SSID (name) を入力。
4  char pass[] = "パスワード"; // WiFiルーターに接続する際の password を入力。
5  int status = WL_IDLE_STATUS; // WiFiの状態
6  IPAddress ip;
7  WiFiServer server(80);
8
9  int IN1 = 3;
10 int IN2 = 9;
```

セットアップ関数にて pinMode の設定とサーバ設立の命令文を追加

```
47 void setup() {
48     Serial.begin(115200);
49     delay(1500);
50
51     //pinModeの設定
52     pinMode(IN1,OUTPUT);
53     pinMode(IN2,OUTPUT);
54
55     //WiFiモジュールのファーム確認
56     String fv = WiFi.firmwareVersion();
57
58     // WiFi接続
59     Serial.print("次のSSIDに接続中: ");
60     Serial.println(ssid); // WiFi接続に使うSSIDを表示
61     status = WiFi.begin(ssid, pass); // WiFi接続
62
63     //接続が確立するまで待機
64     while (status != WL_CONNECTED) {
65         delay(500);
66         Serial.print(".");
67     }
68     delay(10000);
69
70     // 接続完了
71     Serial.println("接続完了");
72     //関数呼び出して接続情報を表示
73     printWifiData();
74     //サーバー設立
75     server.begin();
76 }
```

ループ関数にてまずクライアントの設定を行う。

```
77 void loop() {
78     //サーバの設定と作成
79     WiFiClient client = server.available();
80     if (client) {
81         Serial.println("new client");
82         String currentLine = "";
83         while (client.connected()) {
84             //クライアント情報を取得
85             char c = client.read();
86             //クライアント情報の書き込み
87             Serial.write(c);
88             if (c == '\n') {
89                 if (currentLine.length() == 0) {
90                     // ヘッダー情報
91                     client.println("HTTP/1.1 200 OK");
92                     client.println("Context-type:text/html");
93                     client.println();
94                     // クライアントに記載する情報の設定
95                     // CWのボタンが押されたらAの文字を返す
96                     // CCWのボタンが押されたらBの文字を返す
97                     client.print("<p style='font-size:7vw;'>Click <a href='\"/A\"'>here</a> rotate CW<br></p>");
98                     client.print("<p style='font-size:7vw;'>Click <a href='\"/B\"'>here</a> rotate CCW<br></p>");
99                     client.println();
100                    break;
101                } else {
102                    // then clear currentLine:
103                    currentLine = "";
104                }
105            } else if (c != '\r') {
106                currentLine += c;
107            }
108        }
109    }
110 }
```

クライアントからの文字情報に応じてモータを正転、逆転させる処理を設定する。

```
108
109     // 動作命令の実行:
110     //クライアントからAの文字を受け取ったら
111     if (currentLine.endsWith("GET /A")) {
112         delay(1000);
113         digitalWrite(IN1, HIGH);
114         digitalWrite(IN2, LOW);
115     }
116     //クライアントからBの文字を受け取ったら
117     if (currentLine.endsWith("GET /B")) {
118         delay(1000);
119         digitalWrite(IN1, LOW);
120         digitalWrite(IN2, HIGH);
121     }
122 }
123 // close the connection
124 client.stop();
125 Serial.println("client disconnected");
126 }
127 }
```

実行すると先と同じように IP アドレスなどの情報がシリアルモニタに表示される.

次のSSIDに接続中: AQUOS R9_8385

接続完了

SSID: AQUOSR9_8385

BSSID: A8:D4:19:A7:AB:BA

MAC address: A4:02:9F:FA:12:F4

IP Address: 192.168.236.69

ここで IP アドレスを用いて PC のブラウザのアドレス欄に以下のように入力する.

http://XXX.XXX.XXX.XXX

ここで XXX.XXX.XXX.XXX は上記で表示された IP アドレス.

例の場合は 192.168.236.69 なので http://192.168.236.69 を入力する.

すると以下のようなクライアントページに飛べるので, here の文字をクリックすると設定した A もしくは B の文字が arduino のサーバに送られてモータが制御される.

Click [here](#) rotate CW

Click [here](#) rotate CCW